

AdaBoost (Adaptive Boosting) — Detaylı Konu Anlatımı

Sınav Çalışma Notları

Contents

1. Boosting Mantığı: Zayıf Öğrencileri Sıralı Güçlendirmek	1
2. Zayıf Öğrenici: Karar Kütüğü (Decision Stump)	2
3. En İyi Kütüğü Bulmak: Ağırlıklı Hata (Weighted Error)	2
4. Öğrenici Ağırlığı: Alpha (α) Hesabı	2
5. Örnek Ağırlıklarının Güncellenmesi	3
6. AdaBoost Sınıfı — Fit Döngüsünün Mantığı	3
7. Örnek Ağırlıklarının İterasyonlar Boyunca Evrimi	3
8. $n_{\text{estimators}}$ (Kütük Sayısı) Etkisi	4
9. Hiperparametreler — Sınav İçin Özet Tablo	4
10. AdaBoost'un Diğer Yöntemlerle Karşılaştırması	4
11. Sınavda Sık Sorulan Kavramlar — Hızlı Hatırlatma	5

1. Boosting Mantığı: Zayıf Öğrencileri Sıralı Güçlendirmek

Zayıf öğrenici (weak learner), rastgele tahminden biraz daha iyi performans gösteren basit bir modeldir (örneğin, doğruluğu %50'den biraz yüksek bir model). Boosting'in temel fikri: **çok sayıda zayıf öğreniciyi sıralı (sequential) olarak eğitip, birleştirerek güçlü bir öğrenici (strong learner) elde etmek.**

Random Forest'taki **bagging**'den temel farkı:

- **Bagging (Random Forest):** Modeller **paralel ve birbirinden bağımsız** kurulur; her biri kendi bootstrap örnekleminde eğitilir.
- **Boosting (AdaBoost):** Modeller **sıralı ve birbirine bağımlı** kurulur; her yeni model, bir öncekinin **hatalarına odaklanarak** eğitilir.

Bu fark, iki ailenin neyi azalttığını da belirler: bagging **varyansı**, boosting **yanlılığı (bias)** azaltır.

2. Zayıf Öğrenici: Karar Kütüğü (Decision Stump)

AdaBoost'ta en yaygın kullanılan zayıf öğrenici, **tek bir özellik ve tek bir eşik değerine dayalı, sadece bir bölünme içeren karar ağacı** olan **karar kütüğüdür (decision stump, max_depth=1)**.

Bir kütük şu bileşenlerden oluşur:

- feature_idx: Hangi özellik üzerinden bölünme yapılacağı,
- threshold: Bölünme eşiği,
- polarity: Eşiğin hangi yönünün pozitif/negatif sınıf sayılacağı (+1 veya -1).

Kütük, girdiyi eşiğe göre ikiye ayırır ve her tarafa bir sınıf etiketi atar. Tek başına zayıf olsa da, doğru ağırlıklandırma ile birleştirildiğinde çok güçlü bir topluluk oluşturabilir.

3. En İyi Kütüğü Bulmak: Ağırlıklı Hata (Weighted Error)

Normal bir karar ağacında “en iyi bölünme” saflık ölçütleriyle (entropi/Gini) bulunur. AdaBoost'ta ise her örneğin bir **ağırlığı (w_i)** vardır ve en iyi kütük, **ağırlıklı sınıflandırma hatasını minimize eden** kütüktür:

$$\varepsilon = \sum_{i: \hat{y}_i \neq y_i} w_i$$

Arama süreci:

1. Başlangıçta min_error = ∞ olarak ayarlanır (“şimdiye kadarki en iyi hata”).
2. Her özellik ve her olası eşik değeri denir.
3. Her deneme için ağırlıklı hata hesaplanır; eğer hata %50'den büyükse polarite ters çevrilerek hata 1 - hata yapılır (bir kütüğün “hep yanlış” tahmin etmesi de bilgi taşır — tersini almak onu kullanışlı hale getirir).
4. En düşük ağırlıklı hataya sahip özellik-eşik-polarite kombinasyonu seçilir.

4. Öğrenici Ağırlığı: Alpha (α) Hesabı

Bir kütük eğitildikten sonra, ona **ne kadar güvenileceğini** belirleyen bir ağırlık (α) hesaplanır:

$$\alpha = \frac{1}{2} \ln \left(\frac{1 - \varepsilon}{\varepsilon} \right)$$

Burada eps (küçük bir sabit, örn. 1e-10), $\varepsilon=0$ durumunda bölme hatasını önlemek için hata değerine eklenir (np.clip(weighted_error, eps, 1-eps)).

Alpha'nın Davranışı — Sezgi:

- $\varepsilon \rightarrow 0$ (kütük neredeyse mükemmel) $\rightarrow \alpha \rightarrow +\infty$ (bu kütüğe çok yüksek güven verilir).
- $\varepsilon = 0.5$ (kütük rastgele tahmin ediyor) $\rightarrow \alpha = 0$ (bu kütüğün final tahmine hiç katkısı olmaz).

- $\epsilon \rightarrow 1$ (kütük sürekli yanlış tahmin ediyor) $\rightarrow \alpha \rightarrow -\infty$ (kütüğün tahmini **ters çevrilerek** güvenilir hale getirilir — sürekli yanlış tahmin eden bir model de tersine çevrilince bilgi taşır).

Bu formül, AdaBoost’un “iyi kütüklere daha çok güven, kötü kütüklere daha az (hatta ters) güven” mantığını matematiksel olarak ifade eder.

5. Örnek Ağırlıklarının Güncellenmesi

Bir kütüğün α ’sı hesaplandıktan sonra, her örneğin ağırlığı şu şekilde güncellenir:

$$w_i \leftarrow w_i \cdot e^{-\alpha y_i \hat{y}_i}$$

(Burada y_i ve tahmin $\hat{y}_i$, $\{-1, +1\}$ formatında kodlanmıştır.)

Sezgi:

- Eğer örnek **doğru** sınıflandırıldıysa ($y_i \cdot \hat{y}_i = +1$): ağırlık $e^{-\alpha}$ ile çarpılır $\rightarrow$ **azalır** (bu örnek artık daha az önemli, model onu zaten öğrendi).
- Eğer örnek **yanlış** sınıflandırıldıysa ($y_i \cdot \hat{y}_i = -1$): ağırlık $e^{+\alpha}$ ile çarpılır $\rightarrow$ **artar** (bir sonraki kütük bu örneğe daha çok odaklanmalı).

Güncelleme sonrası tüm ağırlıklar toplamı 1 olacak şekilde **normalize edilir**. Bu, algoritmanın “adaptive” (uyarlanabilir) olmasının özüdür: her iterasyon, bir öncekinin hatalarına göre kendini ayarlar.

6. AdaBoost Sınıfı — Fit Döngüsünün Mantığı

`fit()` metodundaki boosting döngüsü şu adımları `n_estimators` kez tekrarlar:

1. Mevcut örnek ağırlıklarıyla en iyi kütüğü bul (Bölüm 3).
2. Bu kütüğün ağırlıklı hatasından α ’yı hesapla (Bölüm 4).
3. Örnek ağırlıklarını güncelle ve normalize et (Bölüm 5).
4. Kütüğü ve α değerini modele kaydet.

Final Tahmin: Tüm kütüklerin α ile ağırlıklandırılmış oylarının **işareti (sign)** alınır:

$$H(x) = \text{sign} \left(\sum_{m=1}^M \alpha_m h_m(x) \right)$$

Bu, güçlü öğrencilerin (yüksek α) final karara daha fazla, zayıf öğrencilerin daha az etki ettiği anlamına gelir.

7. Örnek Ağırlıklarının İterasyonlar Boyunca Evrimi

Başlangıçta tüm örnekler **eşit ağırlığa** sahiptir ($1/n$). İterasyonlar ilerledikçe:

- Kolay/doğru sınıflandırılan örneklerin ağırlığı **azalır** ve grafikte başlangıç çizgisinin altına düşer.
- Zor/sürekli yanlış sınıflandırılan örneklerin (genelde sınıf sınırına yakın veya aykırı değer olan örnekler) ağırlığı **artar** ve grafikte belirgin şekilde yükselir.

Bu davranış, AdaBoost'un "adaptif" yönünü somut olarak gösterir: algoritma, modelin **zorlandığı bölgelere** giderek daha fazla odaklanır.

Kritik Sonuç — Aykırı Değer Duyarlılığı: Eğer veri setinde yanlış etiketlenmiş bir örnek varsa, bu örneğin ağırlığı her iterasyonda katlanarak artar (çünkü hiçbir kütük onu doğru sınıflandıramaz) — bu da AdaBoost'un **gürültülü veriye ve aykırı değerlere karşı duyarlı** olmasının temel nedenidir.

8. n_estimators (Kütük Sayısı) Etkisi

Random Forest'ta ağaç sayısı arttıkça **varyans** azalıyordu (overfitting riski artmıyordu). AdaBoost'ta durum farklıdır:

- Eğitim doğruluğu genelde **hızla artar** ve yüksek bir seviyede kalır (AdaBoost, eğitim verisine çok iyi uyum sağlayabilir).
- Test doğruluğu da genelde bir süre artar, ancak **çok fazla iterasyon** (özellikle gürültülü veride) test performansını düşürebilir — çünkü model, gürültülü/aykırı örneklerle aşırı ağırlık vermeye devam eder.
- **Pratik sonuç:** AdaBoost, Random Forest'a göre n_estimators seçiminde **daha dikkatli** olunması gereken bir algoritmadır; genelde çapraz doğrulama ile optimum değer bulunur.

9. Hiperparametreler — Sınav İçin Özet Tablo

Parametre	Anlamı	Etkisi
n_estimators	Kütük (zayıf öğrenici) sayısı	Fazlası gürültülü veride overfitting riski taşır
learning_rate	Her kütüğün α katkısını ölçekleyen küçültme katsayısı	Düşükse daha fazla kütük gerekir, genelde daha iyi genelleme
estimator	Zayıf öğrenici tipi	Varsayılan: max_depth=1 karar ağacı (stump)
algorithm (SAMME/SAMME.R)	Çok sınıflı problemlerde kullanılan varyant	SAMME.R olasılık tahminlerini kullanır, genelde daha hızlı yakınsar

10. AdaBoost'un Diğer Yöntemlerle Karşılaştırması

Kriter	Random Forest	AdaBoost
Eğitim stratejisi	Paralel (bağımsız ağaçlar)	Sıralı (birbirine bağımlı kütükler)
Ana amaç	Varyans azaltma	Bias (yanlılık) azaltma
Zayıf öğrenici derinliği	Genelde orta-derin	Genelde çok sığ (stump, max_depth=1)
Örnek ağırlıklandırma	Yok (her bootstrap eşit)	Var (yanlış sınıflandırılanlar ağırlıklandırılır)
Aykırı değere duyarlılık	Düşük	Yüksek
Parallelleştirme	Kolay	Zor (sıralı bağımlılık nedeniyle)

AdaBoost vs Gradient Boosting: İkisi de sıralı boosting yöntemleridir, ancak AdaBoost **örnek ağırlıklarını** güncelleyerek zor örneklerle odaklanırken, Gradient Boosting doğrudan **artıkları (residuals) / negatif gradyanı** tahmin etmeye çalışır. AdaBoost, Gradient Boosting'in üstel kayıp (exponential loss) fonksiyonuna sahip özel bir hâli olarak da yorumlanabilir.

11. Sınavda Sık Sorulan Kavramlar — Hızlı Hatırlatma

- AdaBoost = **Adaptive Boosting**; adaptasyon, örnek ağırlıklarının her iterasyonda güncellenmesinden gelir.
- Zayıf öğrenici genelde **karar kütüğüdür** (max_depth=1).
- $\alpha = 0.5 \cdot \ln((1-\epsilon)/\epsilon)$ formülü: düşük hata $\rightarrow$ yüksek güven; $\epsilon=0.5 \rightarrow \alpha=0$ (katkısız); $\epsilon>0.5 \rightarrow$ negatif α (tersine çevrilir).
- Yanlış sınıflandırılan örneklerin ağırlığı **artar**, doğru sınıflandırılanların ağırlığı **azalır**; ağırlıklar her iterasyonda normalize edilir.
- Final tahmin, tüm kütüklerin α ile **ağırlıklı oylamasının işaretidir**.
- AdaBoost **aykırı değerlere ve gürültülü etiketlere duyarlıdır** — bu, en sık sorulan dezavantaj sorusudur.
- Bagging (RF) **varyansı**, Boosting (AdaBoost) **yanlılığı** azaltır — bu ayrım sınavlarda sıkça karşılaştırma sorusu olarak çıkar.